1) Memory: Characteristics of memory systems

I. Location : Where the memory is present.

Internal Memory : Located inside the computer. These are fast and directly accessible by the processor.

- **Registers** (inside CPU)
- **Cache**
- **Main Memory (RAM)**

External Memory : Outside the CPU, connected through I/O controllers. These are slower but offer large storage.

- **Magnetic Disk (HDD)**
- **Optical Disk (CD/DVD)**
- **Tape**

II. Capacity : How much data the memory can store.

- For **internal memory** → measured in **bytes** or **words**
  (1 word may be 8, 16, 32, or 64 bits depending on system)

- For **external memory** → usually measured in **bytes** (GB, TB)

III. Unit of Transfer : How much data is transferred at a time.

For Internal Memory:

- Usually **word** or a group of words (64, 128, 256 bits)
- **Word** : natural data size used by CPU for instructions and integers
  (ex: 32-bit CPU → 32-bit word)

Addressable Unit

- The smallest unit you can address (might be a byte or a word)
- If address size = A bits → Number of addressable units = 2^A

For External Memory : Data is transferred in **blocks** (large chunks)

IV. Access Method : How memory is accessed.

a) Sequential Access : Data is stored & read in order. To reach record, you must pass through all previous records. A shared read write mechanism is used, and this must be moved from its current location to the desired location, passing and rejecting each intermediate record. Thus, Access time → **very slow & variable.**

b) Direct access : Involves a shared read–write mechanism. However, individual blocks or records have a unique address based on physical location. Mechanism moves to general area, then searches locally by sequential searching, counting, or waiting to reach the final location. Again, access time is variable.

c) Random access : Each addressable location in memory has a unique, physically wired-in addressing mechanism. Access time is **constant.** Thus, any location can be selected at random and directly addressed and accessed. Eg : Main memory and some cache systems are random access.

d) Associative (Content-Addressable) : Data is accessed by **content**, not address. All words are checked simultaneously. Used in **cache memory (tag comparison).** Access time is **constant.** Eg : Cache Memory

V. Performance

a) Access time (latency) : For random-access memory, this is the time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use. For non-random-access memory, access time is the time it takes to position the read–write mechanism at the desired location.

b) Memory cycle time : This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can commence. This additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively. Note that memory cycle time is concerned with the system bus, not the processor.

c) Transfer rate: This is the rate at which data can be transferred into or out of a memory unit. For random-access memory, it is equal to 1/(cycle time)

For non-random access memory : TN = TA + n/R

TN : Average time to read or write N bits
TA : Average access time
n : Number of bits
R : Transfer rate,in bits per second (bps)

VI. Physical Type : Common physical technologies:

- **Semiconductor:** RAM, ROM, Flash
- **Magnetic:** Hard disks,magnetic tapes
- **Optical:** CD/DVD
- **Magneto-optical:** Combines magnetic + optical

VII. Physical Characteristics

a) Volatile vs Nonvolatile

- **Volatile** → data lost when power is off (RAM)
- **Nonvolatile** → retains data (ROM, Flash, HDD)

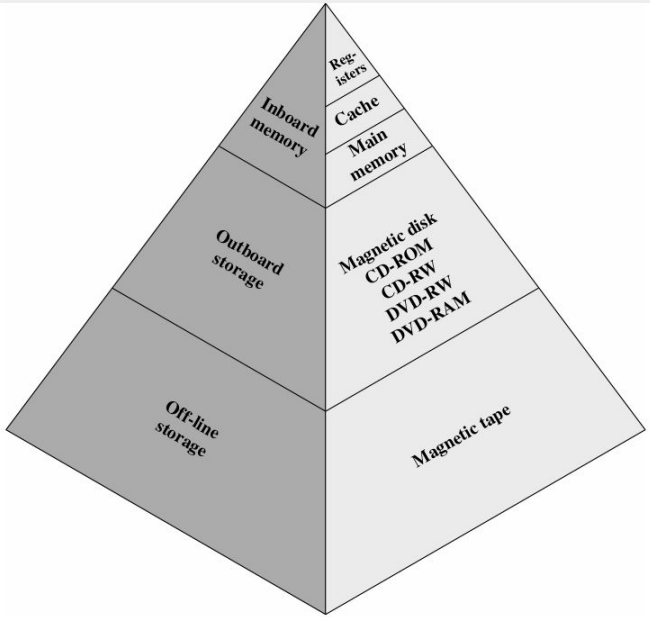
b) Erasable vs Nonerasable

- **Nonerasable** → cannot be changed (true ROM)
- **Erasable** → can be changed electrically (RAM, Flash, EEPROM)

VIII. Organization : How memory bits are arranged into words and stored in modules.

- Determines how many memory chips are needed
- Affects speed and parallel access
- Important for RAM/cache design

2) The Memory Hierarchy :



As one goes down the hierarchy, the following occur:
a. Decreasing cost per bit.
b. Increasing capacity.
c. Increasing access time.
d. Decreasing frequency of access of the memory by the processor.
Thus, smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories. The key to the success of this organization.

Inboard Memory

Registers : The fastest type of memory, directly accessible by the CPU. They hold data and instructions currently being processed.

Cache : A small, high-speed memory that stores frequently accessed data and instructions from main memory. It improves performance by reducing the time it takes to fetch data.

Main Memory (RAM) : The primary storage area for data and programs. It's faster than secondary storage but slower than cache.

Outboard Storage

Magnetic Disk : A secondary storage device that stores data magnetically on a rotating disk. Examples include hard disk drives (HDDs).

Optical Discs : Storage devices that use laser technology to read and write data. Examples include CD-ROM, CD-RW, DVD-RW, DVD-RAM, and Blu-ray discs.

Off-line Storage

Magnetic Tape : A sequential access storage medium used for long-term data storage and backup.

Key points about the memory hierarchy:

I. Speed : The higher in the hierarchy, the faster the memory.

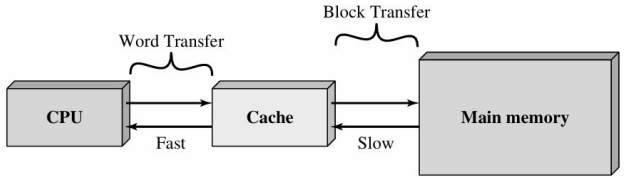
II. Cost : The higher in the hierarchy, the more expensive the memory.

III. Capacity : The lower in the hierarchy, the higher the capacity.

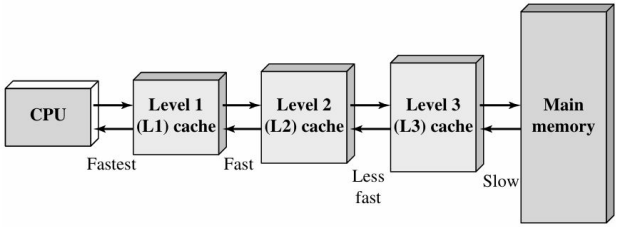
IV. Access Time : The time it takes to access data from a particular level of the hierarchy.

V. Data Transfer Rate : The speed at which data can be transferred between levels of the hierarchy.

3) CACHE MEMORY PRINCIPLES



(a) Single cache



(b) Three-level cache organization

Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

a) Single Cache :

I. Simple structure : The CPU directly interacts with a single cache level.

II. Data transfer : Data is transferred between the CPU and cache in units of words or blocks.

III. Performance : The cache's speed significantly affects the overall system performance.

b) Three-Level Cache Organization :

I. Multiple cache levels : The CPU interacts with multiple cache levels (L1, L2, L3).

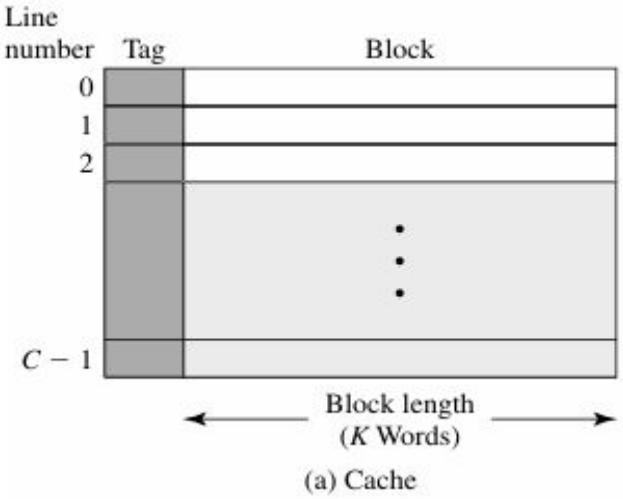
II. Hierarchy : Each level is smaller and faster than the previous one.

III. Data transfer : Data is transferred between levels based on proximity and access patterns.

IV. Improved performance : The three-level hierarchy can improve performance by reducing the average access time for frequently used data.

Cache / Main Memory Structure

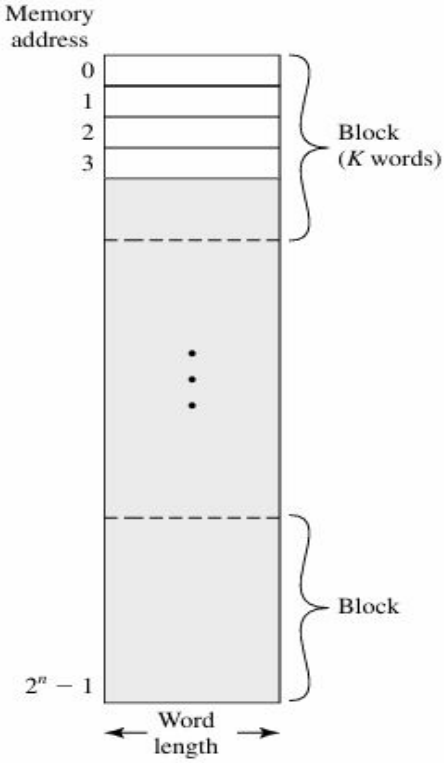
a) Cache :



(a) Cache

I. Line number : Each cache line has a unique number.
II. Tag : A portion of the memory address used to identify the corresponding block in main memory.
III. Block : A contiguous block of data stored in the cache.
IV. Block length : The size of each cache block, typically measured in words.

b) Main Memory :



(b) Main memory

I. Memory address : Each memory location has a unique address.
II. Word length : The size of a single word in memory.
III. Block M-1 : The last block in main memory.

Cache Read Operation

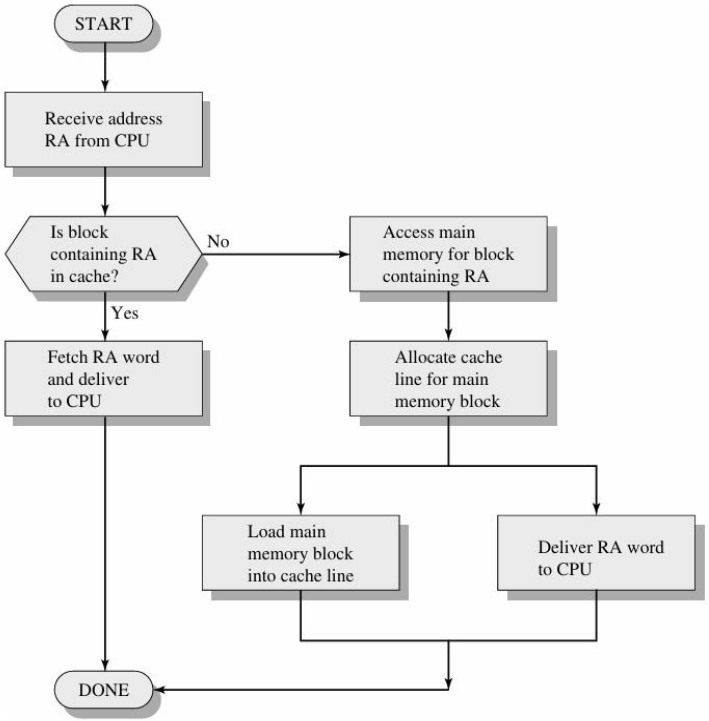


Figure 4.5 Cache Read Operation

1. Receive address RA from CPU : The CPU sends a memory address (RA) to the cache.

2. Is block containing RA in cache?

- **Yes :** If the cache block containing the requested address is already present in the cache, the operation proceeds to step 3.
- **No :** If the block is not in the cache, the operation proceeds to step 4.

3. Fetch RA word and deliver to CPU : The requested word is retrieved from the cache and delivered to the CPU.

4. Access main memory for block containing RA : The main memory is accessed to retrieve the entire block containing the requested address.

5. Allocate cache line for main memory block : A cache line is allocated to store the retrieved block from main memory.

6. Load main memory block into cache line : The retrieved block is loaded into the allocated cache line.

7. Deliver RA word to CPU : The requested word is extracted from the newly loaded block and delivered to the CPU.

Typical Cache Organization

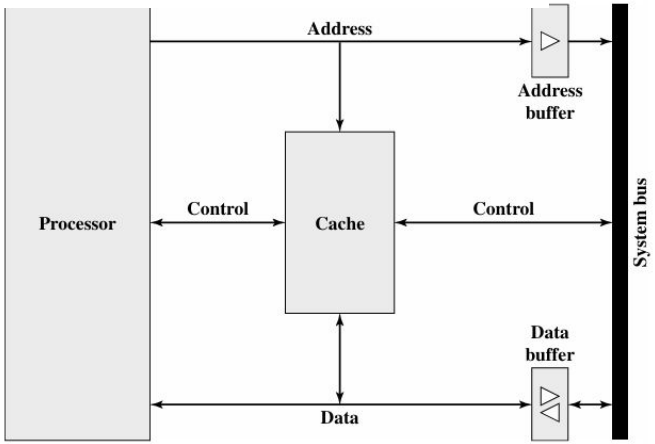


Figure 4.6 Typical Cache Organization

Processor : The central processing unit of the system.

Control : The control unit responsible for managing the cache operations.

Cache : The high-speed memory that stores frequently accessed data.

Address buffer : A buffer that holds the memory address being accessed.

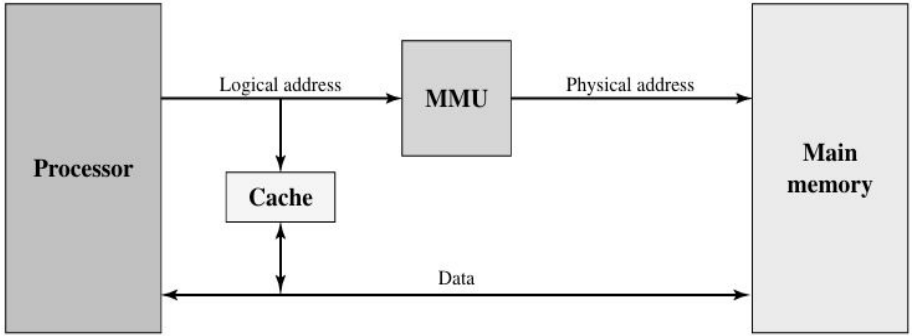
Data buffer : A buffer that holds the data being transferred between the processor and the cache.

System bus : The communication channel connecting the processor, cache, and other components of the system.

4) Elements of cache design

I. Cache Address : Where should the **cache** sit?

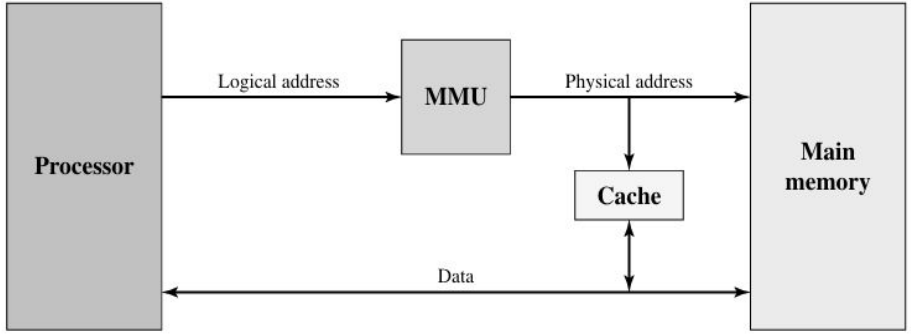
- **Logical Cache**



(a) Logical cache

A logical cache, also known as a virtual cache, Cache placed before the MMU. stores data using virtual addresses. The processor accesses the cache directly, without going through the MMU. It is faster, but problematic during process switching because different processes use identical virtual address spaces.

- **Physical Cache**



(b) Physical cache

A physical cache stores data using main memory physical addresses. Cache placed after the MMU. Physical cache uses physical addresses and avoids the problems faced by logical cache, but introduces a delay because MMU translation must occur before cache access.

II. Cache Size : Total amount of data the cache can store.

- Larger cache → higher hit ratio, but slower access time and higher power
- Small cache → faster but more misses

Cache size must be chosen to balance **hit rate vs. access latency**.

III. Mapping Function : Determines where a memory block will be placed in the cache. Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.

I. DIRECT MAPPING :

The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line. The mapping is expressed as :

i = j modulo m

where :

i = cache line number,
j = main memory block number
m =number of lines in the cache

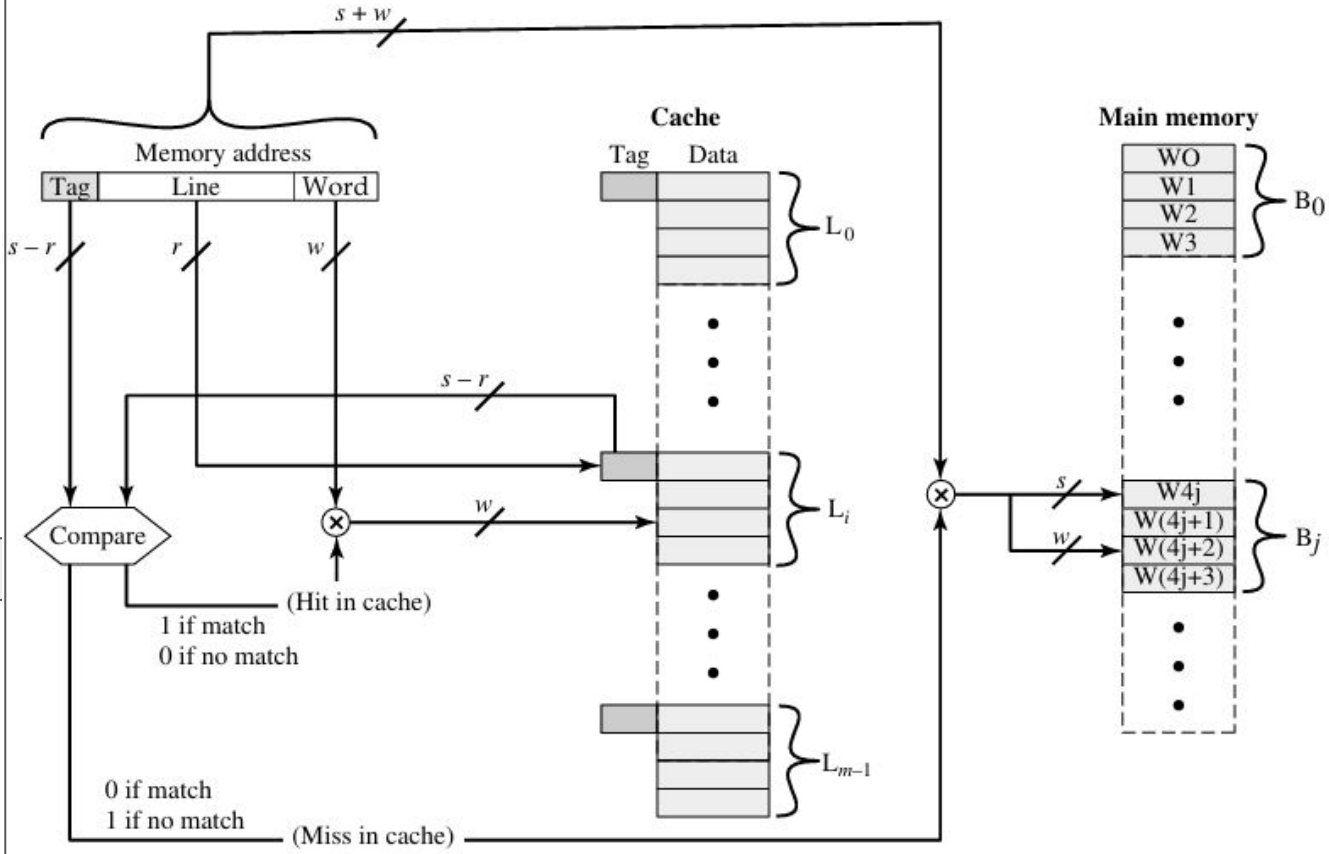


Figure 4.9 Direct-Mapping Cache Organization

■ Address length = (s + w) bits
■ Number of addressable units = 2s+w words or bytes
■ Block size = line size = 2w words or bytes
■ Number of blocks in main memory = 2s+w
2w = 2s
■ Number of lines in cache = m = 2r
■ Size of cache = 2r+w words or bytes
■ Size of tag = (s - r) bits

Components :

■ **Memory address :** The address of the data being accessed.
■ **Tag :** A portion of the memory address used to identify the corresponding cache line.
■ **Line :** A unit of storage within the cache.
■ **Word :** A unit of data (e.g., a byte or a group of bytes).
■ **Cache :** The high-speed memory that stores frequently accessed data.
■ **Main memory :** The primary storage device for data.

Operation :

1. Receive memory address : The cache receives the memory address of the data to be
accessed.

2. Extract tag and line number : The memory address is divided into a tag and a line number.

3. Compare tag : The tag is compared with the tags of all cache lines.

4. Hit / miss :
■ **Hit :** If a match is found, the requested word is retrieved from the corresponding cache line and delivered to the processor.
■ **Miss :** If no match is found, a cache miss occurs, and the requested word is fetched from main memory.

5. Load block : If a miss occurs, the entire block containing the requested word is loaded from main memory into the cache line determined by the line number.

About Direct Mapping :

- Each memory block maps to **exactly one** cache line.
- Uses :
  Index bits → select line
  Tag bits → verify match

Advantages

- Simple
- Fast lookup
- Low hardware cost

Disadvantages

- High conflict misses : different blocks compete for the same line

II. ASSOCIATIVE MAPPING :

It overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of the cachel.

About Associative Mapping :

- Block can be placed in **any cache line**.
- Cache must search **all tags**.

Advantages

- Zero conflict misses
- Highest flexibility

Disadvantages

- Requires associative / parallel search
- Expensive hardware
- Slower lookup

Used in TLBs (Translation Lookaside Buffers).

- Address length $= (s + w)$ bits
- Number of addressable units $= 2^{s+w}$ words or bytes
- Block size $=$ line size $= 2^{w}$ words or bytes
- Number of blocks in main memory $= \frac{2^{s+w}}{2^{w}} = 2^{s}$
- Number of lines in cache $=$ undetermined
- Size of tag $= s$ bits

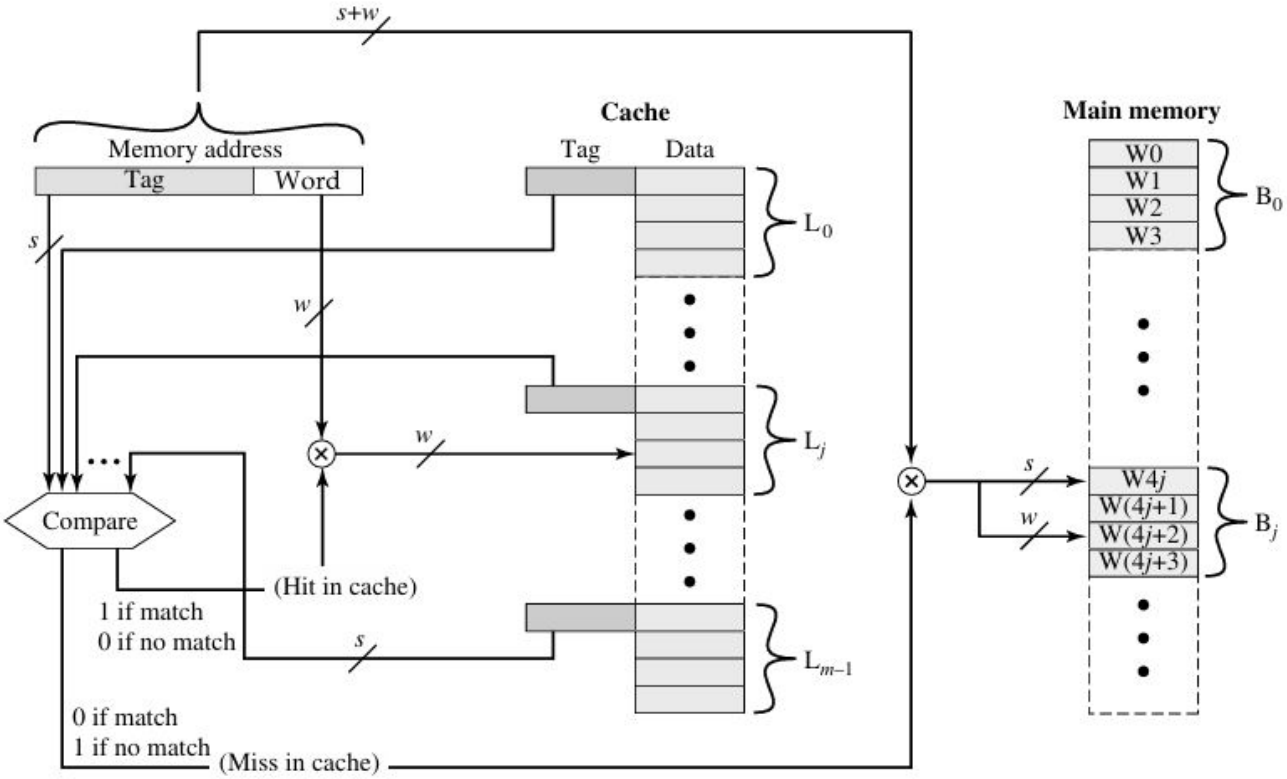


Figure 4.11 Fully Associative Cache Organization

Note that no field in the address corresponds to the line number, so that the number of lines in the cache is not determined by the address format.

Components :

■ **Memory address :** The address of the data being accessed.

■ **Tag :** A portion of the memory address used to identify the corresponding cache line.

■ **Word :** A unit of data (e.g., a byte or a group of bytes).

■ **Cache :** The high-speed memory that stores frequently accessed data.

■ **Main memory :** The primary storage device for data.

Operation :

1. Receive memory address : The cache receives the memory address of the data to be
Accessed.

2. Extract tag : The tag is extracted from the memory address.

3. Compare tag : The tag is compared with the tags of all cache lines.

4. Hit / miss :
■ **Hit :** If a match is found, the requested word is retrieved from the corresponding
cache line and delivered to the processor.
■ **Miss :** If no match is found, a cache miss occurs, and the requested word is
fetched from main memory.

5. Load block : If a miss occurs, the entire block containing the requested word is loaded
from main memory into a vacant cache line.

Set Associative Mapping

It is a compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages. In this case, the cache consists of a number sets, each of which consists of a number of lines.The relationships are

m = v * k
i = j modulo v

Where ,
i = cache set number
j = main memory block number
m = number of lines in the cache
v = number of sets
k = number of lines in each set

This is referred to as k-way set-associative mapping. Direct + Associative.

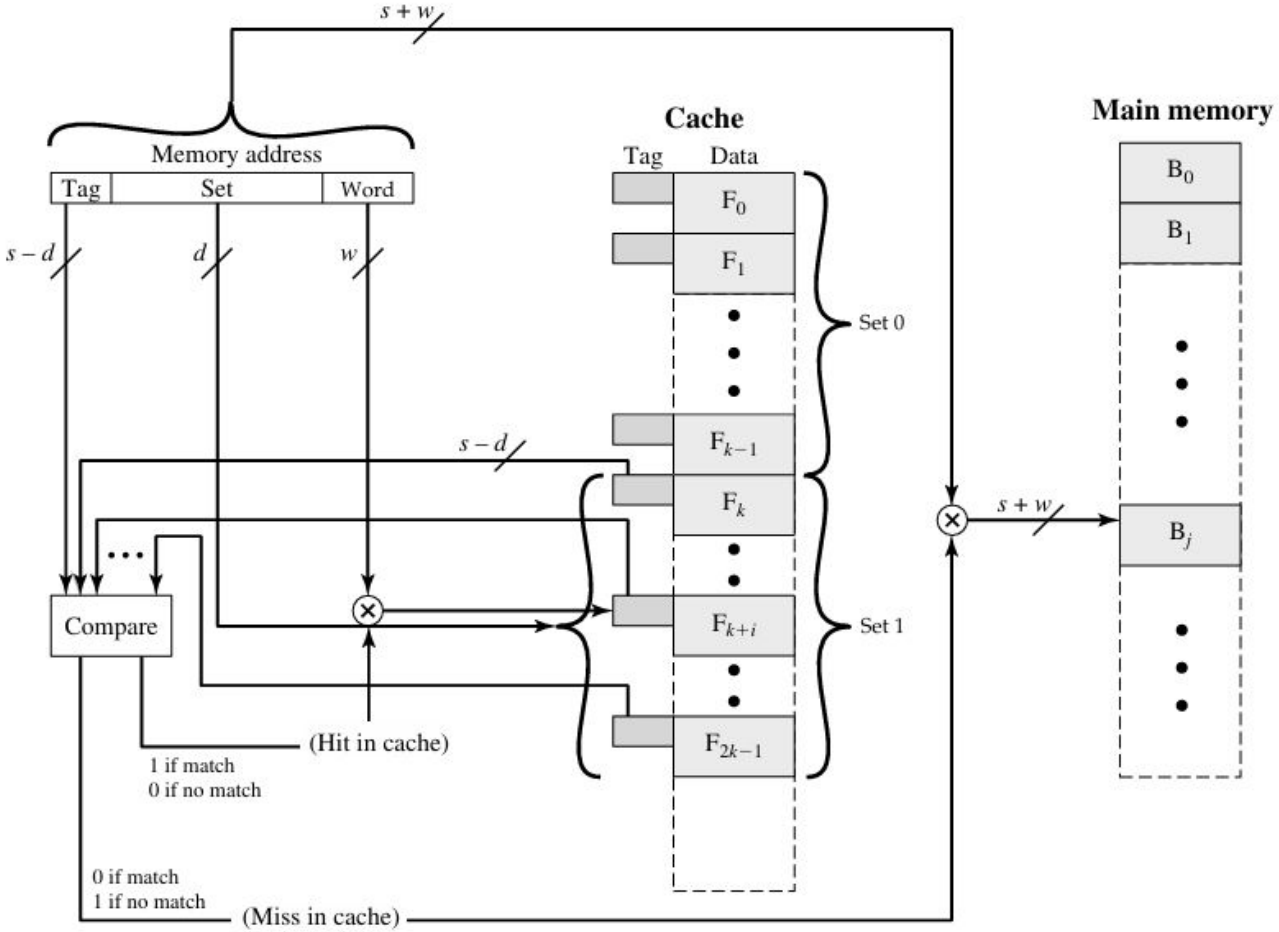


Figure 4.14 *K*-Way Set Associative Cache Organization

- Address length $= (s + w)$ bits
- Number of addressable units $= 2^{s+w}$ words or bytes
- Block size $=$ line size $= 2^{w}$ words or bytes
- Number of blocks in main memory $= \frac{2^{s+w}}{2^{w}} = 2^{s}$
- Number of lines in set $= k$
- Number of sets $= v = 2^{d}$
- Number of lines in cache $= m = kv = k \times 2^{d}$
- Size of cache $= k \times 2^{d+w}$ words or bytes
- Size of tag $= (s - d)$ bits

Components :

■ **Memory address :** The address of the data being accessed.

■ **Tag :** A portion of the memory address used to identify the corresponding cache set.

■ **Set :** A group of k cache lines.

■ **Word :** A unit of data (e.g., a byte or a group of bytes).

■ **Cache :** The high-speed memory that stores frequently accessed data.

■ **Main memory :** The primary storage device for data.

Operation :

1. Receive memory address : The cache receives the memory address of the data to be accessed.

2. Extract tag and set index : The memory address is divided into a tag and a set index.

3. Compare tag : The tag is compared with the tags of all cache lines within the designated set.

4. Hit / miss:

■ **Hit :** If a match is found, the requested word is retrieved from the corresponding cache line and delivered to the processor.

■ **Miss :** If no match is found, a cache miss occurs, and the requested word is fetched from main memory.

5. Load block : If a miss occurs, the entire block containing the requested word is loaded from main memory into a vacant cache line within the designated set.

5) Memory Replacement Algorithms

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced. For direct mapping, there is only one possible line for any particular block, and no choice is possible. For the associative and set associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware.

1. Least Recently Used (LRU) :

Replaces the cache block that has **not been used for the longest time**. Assumes that recently used items will likely be used again → **temporal locality**.

Implementation :

- Order the blocks by time of last use
  Evict the one with the oldest timestamp

Where used?

Common in **small associativity caches** (like L1 2-way or 4-way).

Pros :

- Best approximation of program behavior
- High hit rate

Cons :

- Hardware is expensive, especially for 8-way or 16-way sets
- Needs counters or stacks
- Complex for large associativities

2. First-In First-Out (FIFO) :

Replaces the block that has been in the cache **the longest**, regardless of usage.

How it works?

- Maintain a queue for each set
- Oldest entry is removed first

Used when hardware must be simple and fast.

Pros :

- Very simple hardware
- Easy to implement

Cons :

- Does NOT consider how frequently / recently the block is used
- Can evict a frequently used block

3. Least Frequently Used (LFU) :

Evict the block that has the **fewest number of accesses**.

How it works?

- Each line keeps a counter
- Evict the minimum-count line

Where used?

Rare in CPU caches (too expensive), more common in:

- Virtual memory paging
- Web caches
- Database caching

Pros :

- Good for workloads where frequently-accessed blocks should remain long-term

Cons :

- Extra hardware for counters
- Old blocks with large counts may never get evicted
- Not very practical in real cache hardware

6) Input / Output : I / O Modules

An I/O module contains logic for performing a communication function between the peripheral and the bus.

This module has two major functions (Figure 7.1) :

• Interface to the processor and memory via the system bus or central switch
• Interface to one or more peripheral devices by tailored data links

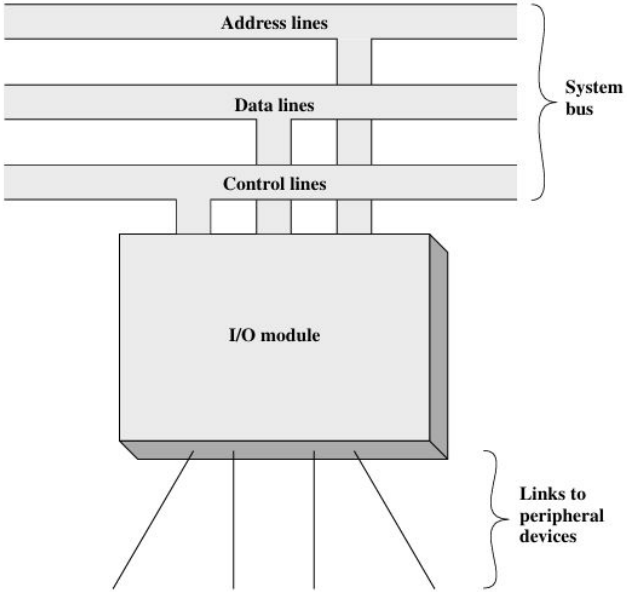


Figure 7.1 Generic Model of an I/O Module

Module Function :

1. Control and timing : To coordinate the flow of traffic between internal resources and external devices. For example, the control of the transfer of data from an external device to the processor might involve the following sequence of steps:

1. The processor interrogates the I/O module to check the status of the attached device.

2. The I/O module returns the device status.

3. If the device is operational and ready to transmit, the processor requests the transfer of data, by means of a command to the I/O module.

4. The I/O module obtains a unit of data (e.g., 8 or 16 bits) from the external device.

5. The data are transferred from the I/O module to the processor.

2. Processor communication :

■ **Command decoding :** The I/O module accepts commands from the processor, typically sent as signals on the control bus. For example, an I/O module for a disk drive might accept the following commands : READ SECTOR, WRITE SECTOR, SEEK track number, and SCAN record ID. The latter two commands each include a parameter that is sent on the data bus.

■ **Data :** Data are exchanged between the processor and the I/O module over the data bus.

■ **Status reporting :** Because peripherals are so slow, it is important to know the status of the I/O module. For example, if an I/O module is asked to send data to the processor (read), it may not be ready to do so because it is still working on the previous I/O command. This fact can be reported with a status signal. Common status signals are BUSY and READY. There may also be signals to report various error conditions.

■ **Address recognition :** Just as each word of memory has an address, so does each I/O device. Thus, an I/O module must recognize one unique address for each peripheral it controls.

3. Device communication :

■ **Data buffering :**

I/O devices are much slower than memory/CPU. The module **buffers data** to match speeds.

- Stores data temporarily in a **data buffer**.
- Prevents CPU from waiting for slow devices.

■ **Error detection :**

I/O devices often produce errors (noise, mechanical faults).
Module handles:

- Parity check
- CRC (Cyclic Redundancy Check) Timeout signals
- Device error reporting

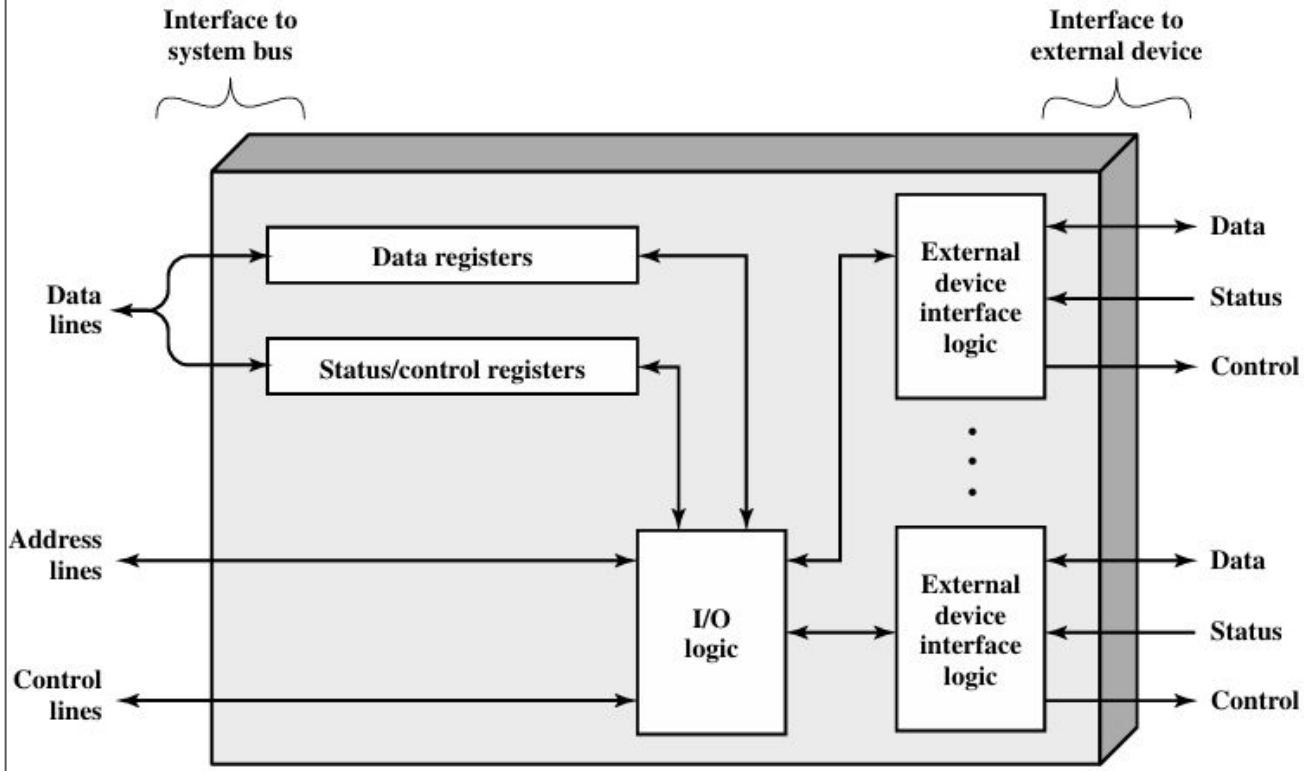


Figure 7.3 Block Diagram of an I/O Module

1. Interface to system bus :

- This component connects the I/O module to the system bus, allowing it to communicate with other components like the CPU and memory.
- It handles the transfer of data, addresses, and control signals between the I/O module and the system bus.

2. Data registers :

- These registers store data that is being transferred between the I/O module and the external device.
- They act as temporary holding places for data during input/output operations.

3. Status/Control registers :

- These registers provide information about the status of the I/O module and the external device.
- They also allow the CPU to control the operation of the I/O module and the external device.

4. I/O logic :

- This component performs the actual input/output operations.
- It controls the flow of data between the I/O module and the external device.
- It also handles the timing and synchronization of input/output operations.

5. External device interface logic :

- This component is specific to the type of external device being connected to the I/O module.
- It provides the necessary interface for communication with the external device.
- It translates the data and control signals between the I/O module and the external device.

6. Data lines :

- These lines carry data between the I/O module and the external device.
- They can be bidirectional, allowing data to flow in both directions.

7. Address lines :

- These lines are used to select a specific device or register within the I/O module.
- They are provided by the system bus and are decoded by the I/O module to identify the target component.

8. Control lines :

- These lines are used to control the operation of the I/O module and the external device.
- They can be used to initiate input/output operations, signal the completion of operations, and indicate error conditions.

7) Programmed I/O :

PIO is a method where the processor directly controls I/O operations. When the processor encounters an I/O instruction, it sends a command to the appropriate I/O module. The I/O module executes the command and updates its status register to indicate the operation's completion. However, the I/O module doesn't actively notify the processor.

The processor must periodically check the status register to determine if the I/O operation is finished.

7) Programmed I/O :

PIO is a method where the processor directly controls I/O operations. When the processor encounters an I/O instruction, it sends a command to the appropriate I/O module. The I/O module executes the command and updates its status register to indicate the operation's completion. However, the I/O module doesn't actively notify the processor.

The processor must periodically check the status register to determine if the I/O operation is finished.

Characteristics

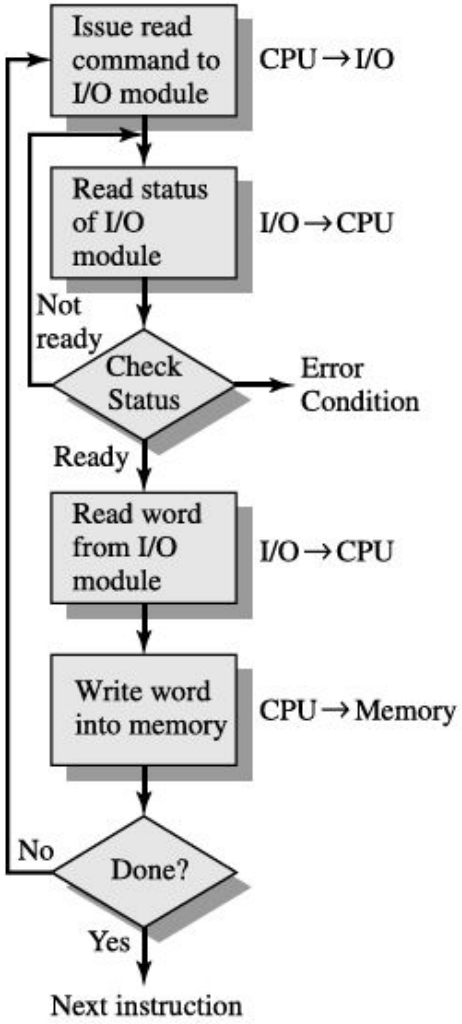
- CPU is **actively waiting** → **polling**, wastes CPU time.
- Simple but inefficient.
- Used in simple devices (keyboard, simple sensors).

Advantages

- Simple to implement.
- No interrupt hardware needed.

Disadvantages

- **CPU is fully involved**, cannot perform other work.
- Lower performance.



(a) Programmed I/O

8) Memory Mapped I/O :

In Memory-Mapped I/O, **I/O device registers are treated as memory locations**. CPU uses **normal load/store instructions** to access I/O devices.

Key Points

- Same **address space** for memory + I/O devices.
- No special I/O instructions.
- Memory addresses are reserved for I/O.

Advantages

- Simple and unified address space.
- Any instruction that works on memory also works on I/O.

Disadvantages

- Reduces available memory address space.
- Requires careful allocation.

Difference Between Memory Mapped I/O & Isolated I/O

Feature	Memory-Mapped I/O	Isolated I/O
Address Space	Same as memory	Separate I/O address space
Instructions	Regular load/store	Special I/O instructions
Complexity	Simple	Slightly complex
Speed	Fast	Slower
Example	ARM, RISC	x86 (IN/OUT instructions)

Isolated I/O : A.K.A **Port-mapped I/O**, is an input/output method where **I/O devices have a separate address space** different from the memory address space.

Key Points :

- CPU uses **special instructions** like `IN` and `OUT` to access I/O ports.
- I/O devices have **their own address range**, separate from memory.
- Needs **special control signals** such as **IORead** and **IOWrite**.
- Reduces address bus conflict because **memory and I/O do not share the same address space**.
- Used in older systems like **x86 architecture (Port 0–65535)**.

9) Interrupt driven I/O :

I/O module **interrupts the CPU** when it is ready, instead of making the CPU poll continuously.

How it works

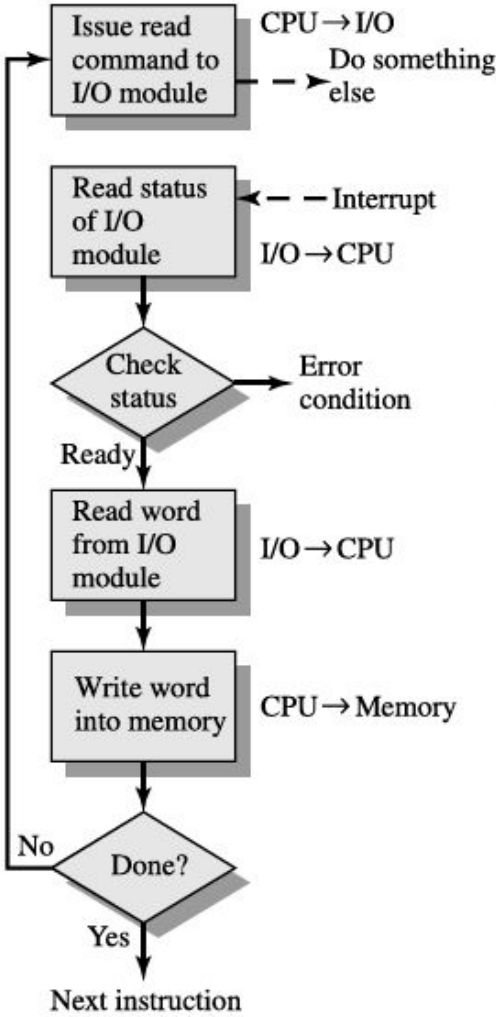
1. CPU issues I/O command or the device issues an interrupt signal to the processor.
2. The processor finishes execution of the current instruction before responding
3. The processor tests for an interrupt, determines that there is one, and sends an acknowledgment signal to the device that issued the interrupt. The acknowledgment allows the device to remove its interrupt signal.
4. CPU executes **Interrupt Service Routine (ISR)**.
5. CPU transfers data and resumes normal execution.

Advantages

- CPU does **not waste time** polling.
- Better performance than Programmed I/O.

Disadvantages

- Interrupt handling has overhead.
- Frequent interrupts can slow down CPU.



(b) Interrupt-Driven I/O

10) Direct Memory Access :

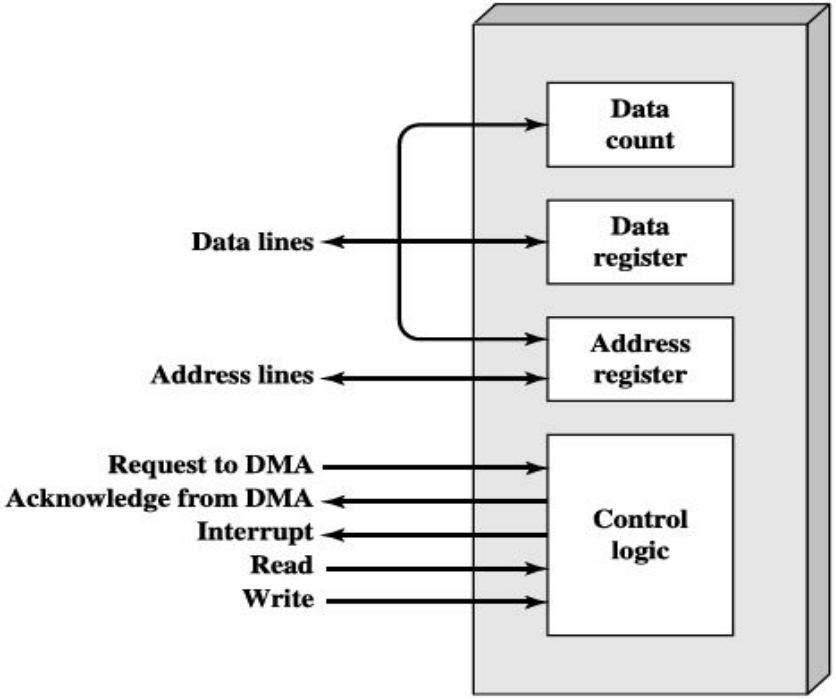


Figure 7.11 Typical DMA Block Diagram

Data Count : Keeps track of the number of data units that need to be transferred between memory and an I/O device. This value is decremented after each data transfer.

Data Register : Temporarily holds the data that is either being read from memory or written to memory during a transfer operation. It acts as a buffer between memory and the device.

Address Register : Holds the address of the memory location where data is to be read from or written to. The address is automatically updated as each transfer occurs.

Control Logic : Manages the overall operation of the DMA controller, handling all signals and coordination between the memory and I/O device. The control logic ensures proper timing and data handling.

Data Lines : These are the pathways through which actual data is transferred between the memory and the device.

Address Lines : These lines carry the memory addresses for data transfer, enabling proper location identification in the memory.

Request to DMA : A signal from the peripheral device to request the DMA controller to start the data transfer process.

Acknowledge from DMA : A signal sent from the DMA controller to the peripheral device, confirming that the request for data transfer has been acknowledged and the operation can begin.

Interrupt : A signal sent to the CPU when the DMA has completed the data transfer process. This frees up the CPU from managing the transfer and allows it to focus on other tasks.

Read and Write : Control signals indicating whether data is being read from or written to the memory. "Read" typically refers to reading from memory to the peripheral, while "Write" refers to writing data from the peripheral to memory.

Questions of Unit 3

Q.1) List and define characteristics of memory system.
Q.2) Explain Direct Mapping with example.
Q.3) Explain I/O module with diagram.
Q.4) Explain typical cache organization with diagram.
Q.5) List advantages of DMA. Show typical DMA.
Q.6) Show cache read operation.
Q.7) List and define characteristics of memory system.
Q.8) Explain Set Associative Mapping with example.
Q.9) Show and explain block diagram of an I/O module.
Q.10) Compare direct, associative and set associative mapping.
Q.11) Explain First In First Out memory replacement algorithm with example.
Q.12) Explain Associative Mapping with example.
Q.13) Explain Interrupt Driven I/O with diagram.
Q.14) Compare memory programmed I/O and memory mapped I/O.
Q.15) Explain Least Recently Used memory replacement algorithm with example.
Q.16) Show and explain memory hierarchy.
Q.17) Explain Direct Mapping with example.
Q.18) Explain Programmed I/O with diagram.
Q.19) Compare memory mapped I/O and interrupt driven I/O.
Q.20) Explain Least Frequently Used memory replacement algorithm with example.